Projet de développement (ou TER)

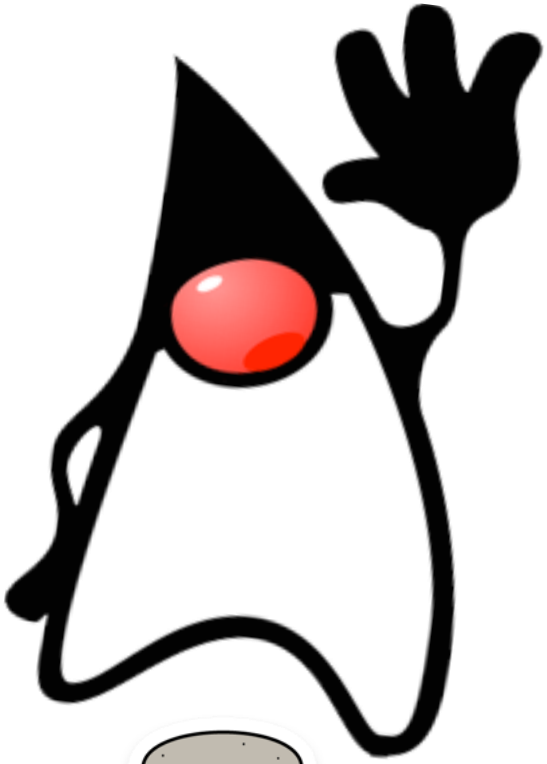
Présentation de SWING

INFO0203

Février 2026

Contact : Jean-Charles.Boisson@univ-reims.fr

JAVA SWING



Support de référence (entre autres) :
SWING la synthèse
Valérie Berthié et Jean-Baptiste Briaud
ISBN 9782100492190

Qu’est ce que SWING ?

- Une bibliothèque :
  - incluse dans toutes les distributions JAVA;
  - servant à produire des IHM portables;
  - étendant la librairie AWT .

⇔ *Abstract Window Toolkit*

AWT

- Bibliothèque qui :
  - Dessine en se servant des ressources graphiques du système
  - Est rapide
  - Peut être (trop) complexe à utiliser

AWT vs SWING

- SWING :
  - Dessine en se servant de la machine virtuelle JAVA
  - Est :
    - une extension d'AWT (et non pas un remplaçant)
    - lente (comparée à AWT)
    - plus souple d'utilisation (qu'AWT)

Pourquoi utiliser Swing ?

- Tout programme JAVA est portable;

Pourquoi utiliser Swing ?

- Tout programme JAVA est portable;
- Toute IHM programmée en JAVA est portable;

Pourquoi utiliser Swing ?

- Tout programme JAVA est portable;
- Toute IHM programmée en JAVA est portable;
- Mais qu'en est-il du rendu graphique ?

Pourquoi utiliser Swing ?

- Rendu graphique :
  - Sous AWT : peut varier fortement d'un système à un autre.

Pourquoi utiliser Swing ?

- Rendu graphique :
  - Sous AWT : peut varier fortement d'un système à un autre.
  - Sous SWING : est globalement stable du fait que c'est la machine virtuelle qui s'occupe du graphisme.

Principe de mise en œuvre

- La mise en forme :
  - Suit le principe de la poupée russe ⇔ objet contenant d'autres objets contenant d'autres objets …
  - Se base sur des « zones » pour ancrer l'information :
    - Globalement une fenêtre ⇔JFrame
    - Localement un « panneau » ⇔ JPanel

Le préfix « J » désigne des objets SWING, les autres sont d'AWT (Frame, …)

Principe de mise en œuvre

- La mise en forme :
  - Nécessite des stratégies de positionnement :
    - Permettant l'ajout dynamique d'éléments
    - Assurant la flexibilité des interfaces ⇔ « responsivité »
    - Fournies en partie par AWT

Principe de mise en œuvre

- Interaction Homme-Machine :
  - Utilisation du patron de conception « Observer »
    - Un objet est observé/écouté par d'autres
    - Tout changement d'état est communiqué aux observateurs/écoutants

Principe de mise en œuvre

- Interaction Homme-Machine :
  - Utilisation du patron de conception « Observer »
    - Chaque action clavier/souris peut être :
      - Détectée
      - Associée à une action spécifique ⇔ fonction/procédure
      - Éventuelle inhibée

Principe de mise en œuvre

- Le principe Modèle Vue Contrôleur (MVC) :
  - Doivent être séparés :
    - La vue ⇔ la partie en interaction avec le client
    - Le modèle ⇔ les données réellement manipulées
    - Le contrôleur ⇔ chef d'orchestre de l'application, il est le lien entre la vue et le modèle

Principe de mise en œuvre

- Le principe Modèle Vue Contrôleur (MVC) :
  - Est difficile à maintenir mais …
  - … assure la pérennité d'une application
  - Demande de bonnes compétences de modélisation orientée objet (et du langage de mise en œuvre)

Mise en œuvre assistée

Créer une interface SWING

- Sans outils ⇔ construction éléments par éléments (voir section suivante)

- Avec outils :
  - Mise en place d'un *design*
  - Ajout des actions
  - Code des actions

Créer une interface SWING

- Sans outils ⇔ construction éléments par éléments (voir section suivante)

- Avec outils :
  - Mise en place d'un *design* — Apache Netbeans
  - Ajout des actions — Apache Netbeans
  - Code des actions — Utilisateur

Avantage de la création assistée

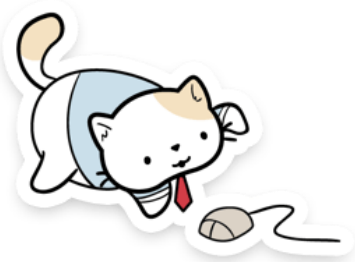
- Permet de :
  - passer outre la gestion des évènements
  - Visualiser directement l'interface
  - découvrir des éléments graphiques par leur utilisation

Désavantages de la création assistée

- Est lent dans la mise en œuvre ⇔ vérification à chaque ajout
- Demande de tout faire à la souris ⇔ le fameux « click-o-drome »
- Génère du code non maitrisé
- Donne une interface approchée

A tester

- Dans Apache Netbeans :
  - Création de 2 projets *Samples >> Java with Ant*
    - *Anagram Game : jeu d'anagramme avec une séparation nette fond/forme*
    - *GUI Form example : création de formulaire spécifique*



LES COMPOSANTS DE BASE

Image par Igor Drondin de Pixabay

Les composants

- Tout composant Swing (JComponent) :
  - est situé dans le paquetage **javax.swing**

    ➔ *dans l'API voir java.desktop (> java 8)*
  - hérite des composants AWT (pour la plupart)
  - est facile à utiliser

Les plus courants

- Le JFrame est :
  - l'extension de l'objet Frame (d'AWT)
  - généralement le conteneur de plus haut niveau :
    - C'est une fenêtre (avec un titre, une taille,…)
  - souvent directement utilisé via héritage :

    *public class monApplication extends **JFrame**;*

Les plus courants

- Le JPanel :
  - sert de conteneur générique à d'autres composants;
  - est souvent le conteneur de plus haut niveau (après la fenêtre) mais n'a pas de rendu graphique;
  - se crée via son constructeur par défaut :

    *var panel = new **JPanel**();*

Les plus courants

- Le JLabel :
  - sert à l'affichage de texte et/ou d'une image.
  - voit son contenu initié à sa création où via la méthode *setText* :

```
var label = new Jlabel(« Hello World »);

label.setText(« Coucou tout le monde »);
```

Les plus courants

- Le JTextField :
  - Sert à la saisie de texte (formulaire);
  - Peut être utilisé pour les mots de passe (contenu caché);
  - Ne gère que des String ⇔ l'obtention des autres types se fait par l'utilisation des objets associés (Integer, Float, …)
  - Est adaptable à son contenu :

```
var textField = new JTextField("Nom Joueur 1");
textField.setColumns(textField.getText().length());
```

Les plus courants

- Le JButton :
  - Est un … bouton pouvant contenir du texte et/ou une image
  - Peut changer d'état suivant le passage de la souris au-dessus de lui.
  - Ne devient utile que lorsqu'un évènement lui est associé,
  - Est facilement créable avec un contenu textuel :

```
var button = new JButton("OK"):
button.setMnemonic('k')// raccourci clavier alt+'k'
```

Les plus courants

- Le JRadioButton :
  - Permet le choix d'un élément parmi plusieurs.
    - Son équivalent en case à cocher existe (JCheckBox).
  - Est généralement associé à un ButtonGroup pour permettre la désélection des autres dès qu'un choix est fait.
  - Peut avoir une valeur par défaut ⇔ lequel est activé

Les plus courants

- Le JRadioButton :

```
var groupe =  new ButtonGroup();

var jRadioButton1 = new JRadioButton("Choix 1");
var jRadioButton2 = new JRadioButton("Choix 2");
var jRadioButton3 = new JRadioButton("Choix 3");

jRadioButton2.setSelected(true);

groupe.add(jRadioButton1);
groupe.add(jRadioButton2);
groupe.add(jRadioButton3);
```

Les plus courants

- Le JComboBox :
  - Permet l'affichage de (petites) listes déroulantes.
  - Propose à l'utilisateur de faire un choix.
  - Peut permettre de saisir une nouvelle valeur.
  - A deux états : ouvert ou fermé.

Les plus courants

- Le JComboBox :

```
String[ ] couleurs = {"Jaune", "Rouge", "Vert"};

var component = new JComboBox<String>(couleurs)
```

Les plus courants

- Le JScrollPane :
  - Conteneur pour consulter un contenu plus grand que le conteneur lui-même
    - ➔ apparition d'une (de) barre(s) de défilement
  - se crée via son constructeur par défaut et l'utilisation de la méthode setViewportView :

```
var jsp = new JScrollPane();
jsp.setViewportView(conteneurAafficher)
```

Les plus courants

- Le JSplitPane :
  - Conteneur permettant de modifier dynamiquement la taille des éléments présents (horizontalement et/ou verticalement)
  - se crée via son constructeur par défaut et l'utilisation de méthode d'ajout selon la place désirée :

```
var jsp = new JSplitPane();
jsp.setLeftComponent(…)
jsp.setRightComponent(…)
…
```

Les plus courants

- Le JTabbedPane :
  - Conteneur permettant d'afficher différents onglets au sein d'une même zone
  - Chaque onglet représente un conteneur particulier
  - se crée via son constructeur par défaut et l'utilisation de méthode d'ajout addTab :

```
var jtp = new JTabbedPane();
jsp.addTab(« nomOnglet1 », conteneur1)
…
```

Exemple à part

- Le JDesktopPane :
  - Conteneur de plus haut niveau simulant l'interface d'un système d'exploitation
  - Permet de gérer plusieurs fenêtres internes à différentes profondeurs de plan.
  - L'exemple est tiré de l'ouvrage de référence (p.58)

Paramétrer les composants

- Tout composant est :
  - Visible ou non :

    setVisible*(true/false)*

    boolean ***isVisible****()*
  - Activé ou non :

    setEnabled*(true/false)*

    boolean ***isEnabled****()*

Paramétrer les composants

- Taille des composants :
  - Taille réelle et taille souhaitée :
    - Historiquement dépendent d'un objet Dimension (objet AWT):
      - *Dimension* ***getSize****()* *
      - *Dimension* ***getSize****(Dimension dim)*
      - Accesseur **getHeight()** et **getWidth()** sur Dimension
    - La taille peut être imposée en dur

      ➔ ***setSize****(Dimension)*
    - La taille peut être calculée en fonction des contenus correspondant.
    - Il est possible de directement faire un set(Height/Width)

Paramétrer les composants

- Modifier le curseur suivant les composants :
  - Croix, sablier, …

- Associer un curseur prédéfini à un composant :

```
var jbutton = new JButton("OK");

var cursor = Cursor.getPredefinedCursor(Cursor.CROSSHAIR_CURSOR);

Jbutton.setCursor(cursor);
```

Paramétrer les composants

- Utiliser des bulles d'aide (JToolTip):
  - Permet d'améliorer l'utilisation de l'interface
  - Est compatible avec tous les composants
  - Méthode générique : *setToolTipText*("De l'aide")
  - Méthode plus spécifique pour certains composants (panneau à onglets par exemple)

Paramétrer les composants

- On peut jouer sur les couleurs des composants :

  setBackground*(Color)*

  setForeground*(Color)*

- On peut donner des bordures particulières aux composantes (pour grouper différentes éléments) :

  setBorder*(Border)*

Color est un objet AWT mais Border est un objet SWING

LES DISPOSITIONS

Image par [200 Degrees](#) de [Pixabay](#)

Disposition des composants

- Par défaut :
  - un composant s'ajoute à celui qui le contient
    ➔ méthode *add*
  - l'ordre d'ajout impacte directement sur l'ordre d'affichage.
  - Tous les composants suivent un même *layout*

Qui contient tout ?

- Le composant qui contient tous les autres est (dans bien des cas) le JFrame.

- Un objet JFrame :
  - Est une fenêtre.

  - Possède :
    - Un titre.
    - Une barre de menu (JMenuBar).
    - Un comportement par défaut lors de sa fermeture.

Un *layout* c'est quoi ?

- C'est une stratégie de positionnement des composants (disponible dans AWT)
- JAVA fourni des *layout* par défaut.
- On associe un *layout* à un composant avant d'inclure des composants à l'intérieur.
- Le *layout* par défaut est le FlowLayout (centré)

Le FlowLayout

- *Layout* similaire au comportement des composants sans *layout* *(à la différence que l'on peut gérer l'alignement à gauche et à droite)*

- Tout composant qui est ajouté l'est de :
  - Gauche à droite
  - De haut en bas

- La taille du composant impacte sur l'agencement des composants internes.

Le GridLayout

- *Layout* pour la disposition matricielle;
- Correspond à une grille de positionnement;
- L'ordre des éléments se fait comme dans le FlowLayout;
- Un redimensionnement de la fenêtre conserve l'ordonnancement des composants.

Le GridLayout

- La déclaration requiert les dimensions de la grille.

- Il faut fixer le *layout* avant d'ajouter des composants.

- Exemple :

```
var layout = new GridLayout(3,2);
monComposant.setLayout(layout);
```

Le BorderLayout

- *Layout* similaire au déploiement de site web*

- Respecte ce schéma de positionnement :

<table>
<tr><td colspan="3">North</td></tr>
<tr><td>West</td><td>Center</td><td>East</td></tr>
<tr><td colspan="3">South</td></tr>
</table>

Le BorderLayout

- L'ajout d'un composant se fait par défaut au centre ou sinon à une autre position à préciser (c'est le *layout* par défaut des JFrame).

- Préserve l'ordre des composants en cas de redimensionnement.
- Exemple :

```
monComposant.setLayout(new BorderLayout());
monComposant.add(unComposant)
monComposant.add(autreComposant,BorderLayout.NORTH);
```

Le Boxlayout

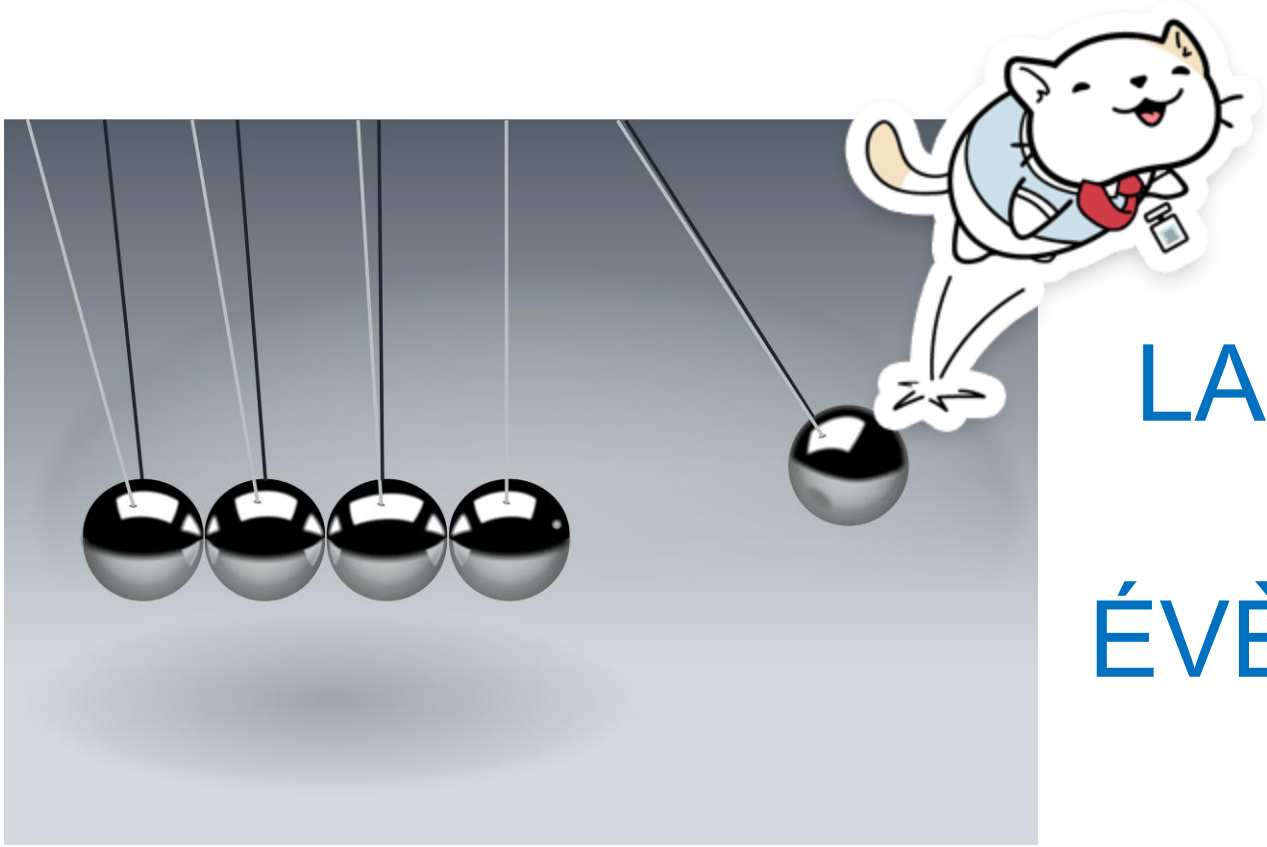
- *Layout* (Swing) plus rigide que les autres;
- Impose l'alignement (vertical ou horizontal);
- Association (au sens du génie logiciel) entre le *layout* et le composant concerné.
- Exemple :

```
var layout = new BoxLayout(monComposant,BoxLayout.X_AXIS);
monComposant.setLayout(layout);
```

Les autres layout

- GridBagLayout : il est entièrement paramétrable et donc par conséquent moins aisé d'utilisation.

- FormLayout :
  - Non inclus (directement) dans JAVA
  - Exemple de *layout* mis en place pour un type d'agencement en particuliers : les formulaires.



LA GESTION DES ÉVÈNEMENTS

Image par [Michal Jarmoluk](#) de [Pixabay](#)

Ecouter et agir

- Toute action repose sur le principe d'écoute et de réaction.
- Un composant est écouté et si l'écouteur est prévenu d'un évènement, il essaye de le gérer s'il en est capable.
- Les ActionListener et les ActionEvent.

Gérer des évènements

- Il faut implémenter le *listener* capable de gérer l'évènement concerné (clic de souris, touche au clavier, déplacement souris, …)

- Leur type général est java.awt.event

- Exemple de listener :

  ➔ MouseListener : clic, entrée, passage ou sortie de la souris d'une zone, bouton maintenu/relâché.

Gérer des évènements

- Les Listener sont des interfaces, il faut :
  - Créer une classe les implémentant
  - Implémenter toutes les méthodes associées (même si une seule nous intéresse ☹)
  - Associer la nouvelle classe au composant concerné

➔ et hop c'est prêt ☺

Gérer un évènement

- Écrire la classe qui gère l'évènement :
  - Exemple : quitter une application ⇔ couper la machine virtuelle
  - Choisir un nom, par exemple Exit.
  - Si un composant a cette classe comme action associée ➔ un évènement sur ce composant provoquera la fin du programme.

Exit.java

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

class Exit implements ActionListener
 {
   public void actionPerformed(ActionEvent ev)
        {
                System.exit(0);
        }
 }
```

Utilisation de *Exit*

- Association au choix Quitter d'une IHM.

- Utilisation de la méthode addActionListener

jmi_Quitter.addActionListener(new Exit ());

JMenuItem

Gérer un évènement

- Écrire une classe à part :
  - Utile quand l'action est utilisée par plusieurs classes
  - Contraignant si l'action n'est utilisée que dans une classe mais plusieurs fois

  - Encore plus contraignant si l'action n'est utilisée qu'une seule fois dans une seule classe

Gérer un évènement

- Action utilisée plusieurs fois dans une classe :
  - Seule cette classe a besoin de cette action
  - Aucune nécessité que cette action soit accessible en dehors.

    ➔ classe interne

Les classes internes

- Non spécifiques au traitement des évènements
- Déclarées à l'intérieur d'une classe au même niveau qu'une méthode
- Ont le type spécifique de la classe englobante.
- Ont accès aux variables (même privées)

Gérer un évènement

- Action utilisée une seule fois dans une classe :
  - Seule cette classe a besoin de cette action
  - Aucune nécessité que cette action soit accessible autre part même au sein de la classe.
  - Utilité d'un nom ???

    ➔ classe anonyme

Une classe anonyme

- Elle est :
  - Interne à une classe
  - Définie seulement par son contenu (pas de nom)
  - Dans le cas d'évènements, elle permet d'implémenter une interface en la citant.

Exemple

- Exemple d'une action liée à un bouton de validation : boutonValider.

- C'est une action liée uniquement à ce bouton.

```
boutonValider.addActionListener(new ActionListener()  {
        public void actionPerformed(ActionEvent ev) {
                //traitement
        }
});
```

Exemple

- On peut condenser l'écriture grâce à une expression lambda
- Le code :

```
boutonValider.addActionListener(new ActionListener()  {
        public void actionPerformed(ActionEvent ev) {
                //traitement
        }
  });
```

- Devient :

```
boutonValider.addActionListener( ev -> //traitement) ;
```

Résultats de compilation

- Classe « MaClasse » ➔ *MaClasse.class*

- Classe « MaClasse » + classe interne « Internal »
  - ➔ *MaClasse.class*
  - ➔ *MaClasse$Internal.class*

- Classe « MaClasse » + une classe interne anonyme
  - ➔ *MaClasse.class*
  - ➔ *MaClasse$1.class*